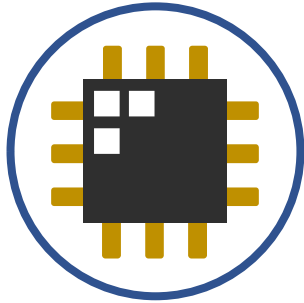




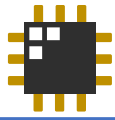
EEE 113:

Introduction to Electrical and Electronics Engineering Systems

Week 1 – Logic gates, Boolean algebra,
and logical equivalence

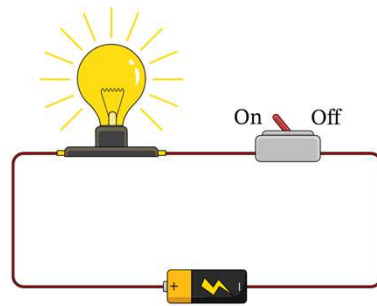
1

This is Week 1 part 2. We will be covering Boolean algebra and logical equivalence



Introduction to logic

- Let's utilize binary bits and do something cool with them



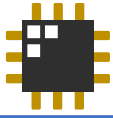
2

Binary bits are incredibly powerful because they can represent a vast range of information using only two states - 0 and 1. These two states can correspond to anything: on/off, true/false, yes/no, etc. This makes them extremely versatile.

On its own, it might not be much. This is just a simple circuit. However, if we consider a bunch of them, then we can relay some information. Imagine we have 8 such circuits side by side. Each circuit can independently be on or off. This means, we technically have an 8-bit binary string!

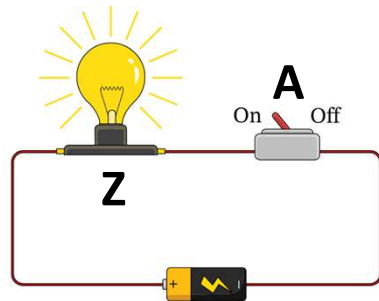
This gives us $2^8 = 256$ different combinations of on and off states across these 8 circuits. That's a lot of information that can be represented using just 8 binary bits!

In fact, this is how computers work. They use millions of tiny electronic circuits (transistors) that can be switched on or off. By manipulating these on and off states in complex ways, computers can perform calculations, store information, and much more.



Introduction to logic

- **Switches:** Basic element of a physical circuit
- If A is 1, switch is closed, Z (light bulb) turns on
- If A is 0, switch is open, Z (light bulb) turns off



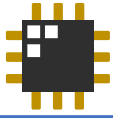
$$Z = A$$

Whatever the state of A is,
Z copies it

3

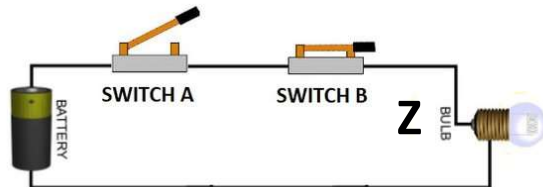
Now, let's consider this simple circuit with a switch and a light bulb. The switch can be in one of two states: on (1) or off (0).

The light bulb will light up when the switch is on (1) and will be off when the switch is off (0). In this case, the output, which is the light bulb represented as Z, exactly depends on the input, which is the switch represented as A. Thus, we can say that $Z = A$ in this example. That is an expression that we can use to describe this setup.



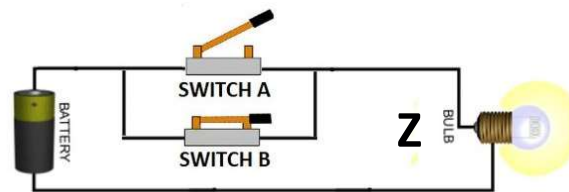
Introduction to logic

- We can make more complex ones!



$$Z = A \text{ AND } B$$

A and B needs to be 1 so that Z turns on (1)



$$Z = A \text{ OR } B$$

A or B needs to be 1 so that Z turns on (1)

Image credit: "How were computers made using very simple logic gates..." quora.com

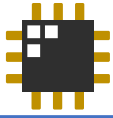
4

The previous circuit is not really interesting since the output just copies the input. However, we can include more switches and play around with the configuration for a bit.

Consider the first circuit at the top. Imagine you have two light switches in a row (series). Both switches need to be in the 'ON' position (1) for the light bulb at the end of this series circuit to light up. If either of the switches is in the 'OFF' position (0), even just one, the light bulb will not light up. We can write an expression describing this scenario. The light bulb is still the output Z and now we have two inputs. For the light bulb, Z, to turn on, both A AND B must be on. We have that AND condition there. That's one of the basic logic component being used in a computer system!

Now consider the second circuit at the bottom. Imagine you have two light switches in parallel. In this case, the light bulb will light up if either one of the switches or both are in the 'ON' position (1). The light bulb will only be off if both switches are in the 'OFF' position (0). So now, what is the expression for this one? For the light bulb, Z, to turn on, either A OR B must be on. We have that OR condition there. That is another basic logic component being used in

the world of computing.



Introduction to logic

- Early computer designers built ad hoc circuits from switches
 - They began to notice common 'patterns'/logic in their work: ANDs, ORs
- Claude Shannon's 1938 thesis made link between switches and logic.
 - Shoutout to the Comms section of EEE 113!
- George Boole came up with a linguistic algebra to encode logic into a language that can be manipulated and solved mathematically
 - We call it *Boolean logic* in his honor.

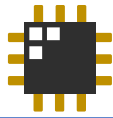
5

Now, before we proceed, a little bit of history just for added appreciation. Why are we presenting these switch analogy and AND and OR logic? The development of early computers involved building ad hoc circuits from switches. As these early computer designers worked, they began to notice common patterns or logic in their work, such as ANDs and ORs. These patterns are fundamental to how we process and interpret information.

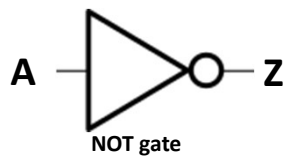
Claude Shannon, in his groundbreaking 1938 thesis, made the crucial link between these physical switches and abstract logic. This was a revolutionary idea that laid the foundation for digital circuit design. Shoutout to the Communications section of EEE 113, since I believe his name was also mentioned there.

Around the same time, George Boole was developing a linguistic algebra to encode logic into a language that could be manipulated and solved mathematically. This became known as Boolean logic, named in his honor. Boolean logic is the backbone of digital electronics and computing, allowing us to represent complex logical statements in a form that machines can understand and process.

These developments were pivotal in the evolution of computing, transforming it from a niche scientific tool into a ubiquitous part of modern life. The principles of AND, OR, and Boolean logic that were recognized and formalized by these pioneers are still fundamental to how our digital devices operate today.



Introduction to logic



NOT gate

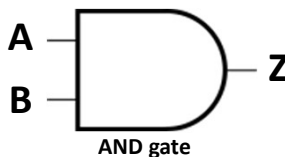
$$Z = A'$$

We usually say "A prime", or "not A"

A	Z
0	1
1	0

These are what we call **truth tables**

What is the output given the input/s combination?

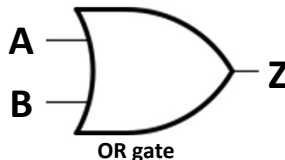


AND gate

$$Z = AB$$

We usually say "A and B"
In a way it is like binary multiplication, right?

A	B	Z
0	0	0
0	1	0
1	0	0
1	1	1



OR gate

$$Z = A + B$$

We usually say "A or B"
Don't be fooled, this is not addition!

A	B	Z
0	0	0
0	1	1
1	0	1
1	1	1

Image credit: wikimedia.com

6

Now, let's go back. Here we have the three basic logic gates in digital circuits. The last two is something that you're already familiar about, AND and OR. The first one is what we call the NOT logic, which is basically just the inverse of whatever the input is.

On the left, we see the logic gate diagrams of the three. For the NOT gate, also called an inverter, it is just a triangle with a circle at the end. The circle at the end is the most important thing here, since that implies the negation, or the inversion of the input. AND gate looks like the one in the middle and the OR gate is the one below. Remember how their diagrams are drawn as they will show up frequently, especially as we move towards more complex circuits.

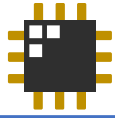
In the middle column, we have the equivalent logic statements that describes their functionality, together with the symbols we use to represent them. For a NOT gate, the output Z is just A prime. The prime there implies the inversion of whatever A is. The truth table, which describes the functionality of the logic is shown on the right. As you can see, Z is just the inverted version of A.

For the AND gate, it is the output Z being equal to A and B. We usually just write them together like this, as if multiplying them in math. Actually, if you think about it,

the AND logic is indeed similar to a 1-bit multiplication, don't you think? Looking at the truth table, the output Z only becomes 1 if both A and B are 1. That's also similar to a 1-bit binary multiplication, indeed.

For the OR gate, it is the output Z being equal to A or B. We usually write the OR operation like this, which is, yes, can be misleading. This is not an addition when we are discussing logic gates. This operator is the OR operator. Looking at the truth table for the OR gate, as described previously, the output Z becomes 1 if either A or B becomes 1.

We typically use truth tables to describe the functionality of a circuit. For more complex logic, it also allows us to derive what the equivalent logical expression should be.



Introduction to logic

- Boolean logic is actually just set theory!

- Can you identify the logic being implied in the following diagrams?
(shaded means true/1)

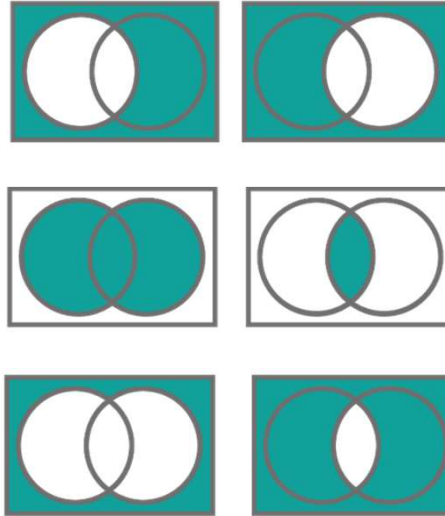
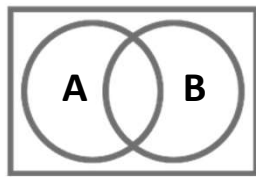


Image credit: "Bit arithmetic and Venn Diagrams" steemit.com

7

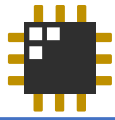
Another perspective in Boolean logic is set theory. The NOT, AND, and OR logic can be imagined in the way of Venn diagrams as well.

Look at these diagrams, assuming A is the left circle and B is the right circle inside the box, would you be able to identify the corresponding logic shown in the diagram?

(top left) A' (top right) B'

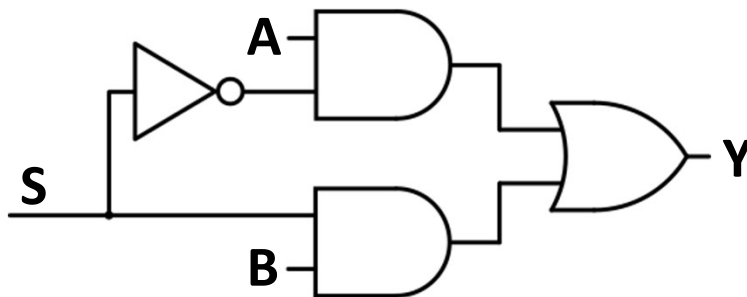
(mid left) $A+B$ (mid right) AB

(bottom left) $(A+B)'$ (bottom right) $(AB)'$



More complex logic

- We can make combinations of logic gates to make more complex logic!



S	A	B	Y
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

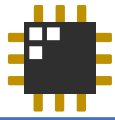
8

From the previous diagram, we figured that we can actually chain logic to create more complex ones.

Here's a rather complicated digital logic circuit, but everything you see are just the logic gates that you've seen before.

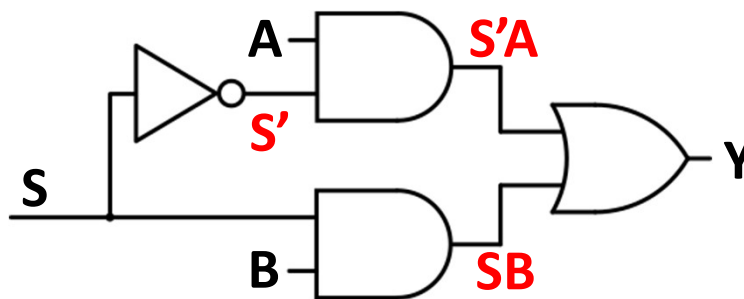
We have three inputs now, A, B and S. Since we have three inputs, each input can be either 0 or 1 independently of one another. Therefore, we have $2^3 = 8$ different possible combinations. The truth table here writes all the different combinations of the three inputs. As you can see, it is actually written in the order of increasing binary value. That's one way to guarantee that you have covered all the different combinations.

Now, let's try to write the equivalent Boolean expression of this logic circuit:



More complex logic

- We can make combinations of logic gates to make more complex logic!



S	A	B	Y
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

$$Y = S'A + SB$$

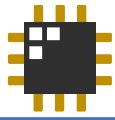
9

From the input S, we identify the leftmost gate as the NOT gate, so the output of the NOT gate is simply the inverted version of what the input is. So in this case, the output is S' since the input was S.

Moving to the right, the output goes into another gate, we identify this as the AND gate, therefore we simply write the AND of both inputs, so in this case, we have both A and S' as inputs, so we AND them together and we end up with S'A. We can also write it as AS', but we'll discuss more about those properties later.

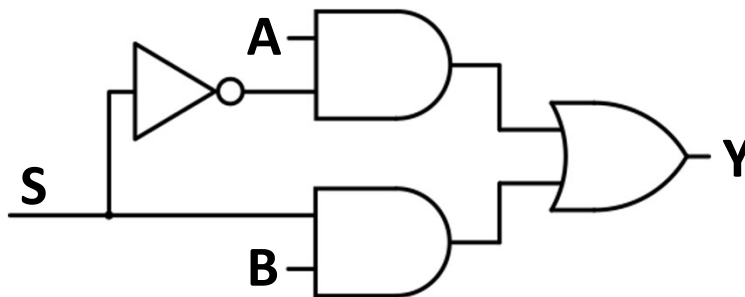
Down the bottom part of the circuit, the input S also goes into another logic gate. This one is the same gate as earlier but with different inputs. We AND both S and B together, so the output is SB.

Finally, the rightmost gate is where the outputs of the AND gates are connected. We have the OR gate here where the output Y is connected to. Therefore, we simply OR the two inputs, which are these expressions, and we can write $Y = S'A + SB$.



More complex logic

- We can make combinations of logic gates to make more complex logic!



S	A	B	Y
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

$$Y = S'A + SB$$
$$(1)A + (0)B$$

10

Then, let's complete the truth table by writing out the values of the output Y for every combination of S, A and B. You can approach this in two ways: either test all the possible input combinations in the circuit and see what the output should be for every combination; or look at the equivalent expression and see if you can complete the table from there.

$$Y = S'A + SB$$

We can only look for the cases where Y becomes 1.

If S = 1:

Then S' (the inverted version of S) is 0.

So, S'A is 0 ANDed with A.

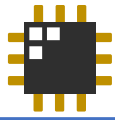
Looking at the AND logic, both inputs must be 1 for the output to be 1.

So, if one of the inputs is 0 (which is the case here), then the output will be 0.

Therefore, S'A will be 0 for all values of A.

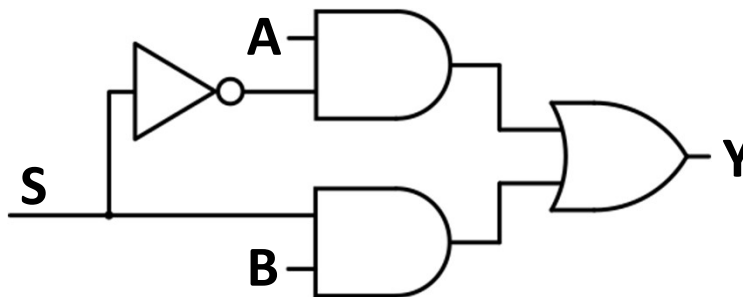
However, since S is 1, SB will be B for all values of B.

So, Y = S'A + SB will be 1 when B is 1.



More complex logic

- We can make combinations of logic gates to make more complex logic!



S	A	B	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	
1	0	1	
1	1	0	
1	1	1	

$$Y = S'A + SB$$
$$(0)A + (1)B$$

11

If $S = 0$:

Then S' is 1.

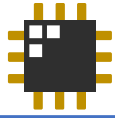
So, $S'A$ is 1 ANDed with A.

Again, looking at the AND logic, both inputs must be 1 for the output to be 1.

So, $S'A$ will be 1 when A is 1.

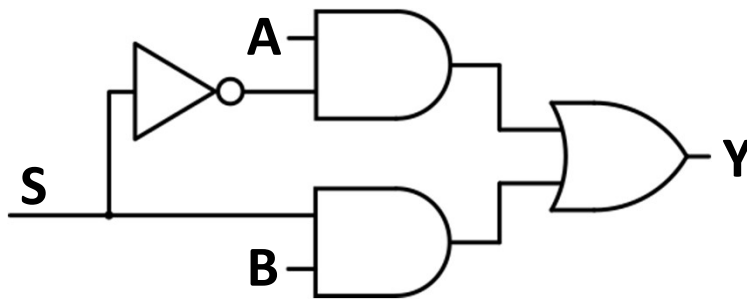
However, since S is 0, SB will be 0 for all values of B.

So, $Y = S'A + SB$ will be 1 when A is 1.



More complex logic

- We can make combinations of logic gates to make more complex logic!



$$Y = S'A + SB$$

S	A	B	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

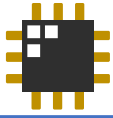
12

So, in summary, Y will be 1 in two cases:

When S is 1 and B is 1.

When S is 0 and A is 1.

This allows us the complete the truth table for this circuit now.



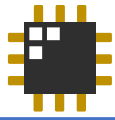
Truth tables to Boolean expression

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

- Boolean expressions and truth tables represent the same thing
 - You can derive one from the other and vice versa!
- Write the equivalent Boolean expression for the truth table on the left.
 - **Tip:** Write the conditions that needs to be satisfied to make the output 1

Boolean expressions and truth tables are two different ways of representing the same logical relationships. They are interconnected and you can derive one from the other.

To derive a Boolean expression from a truth table, you can follow these steps:
1. Identify the rows in the truth table where the output is 1.



Truth tables to Boolean expression

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

A'B'C'

_____?

_____?

_____?

- For the first row, we say **Q** becomes 1 if A is 0 AND B is 0 AND C is 0

A' AND B' AND C'

- If something *needs to be 0* to satisfy a condition, that implies the **NOT** logic

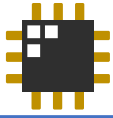
- If a *specific combination of inputs* need to be satisfied, that implies the **AND** logic

- Looking at a single row

2.) For each of these rows, write down a term that consists of each input variable. If the variable is 1 in this row, write it down as it is. If it's 0, write down its complement (i.e., with a NOT operation).

3.) Looking at a single row in the truth table, a specific combination of inputs that needs to be satisfied implies the logical AND. For example, in this first row. Q = 1 if and only if, A is 0, and B is 0, and C is 0. So all of them must be 0 to make the output Q to be 1.

So if you write that in Boolean expression form, it is literally the English statement earlier. Q=1, if A is 0 AND B is 0 AND C is 0. And there's your expression: A' AND B' AND C'. This is for the first row.



Truth tables to Boolean expression

A	B	C	Q	
0	0	0	1	$A'B'C'$
0	0	1	1	$A'B'C$
0	1	0	0	
0	1	1	0	
1	0	0	0	
1	0	1	1	$AB'C$
1	1	0	0	
1	1	1	1	ABC

- If *either of the several combinations of inputs* can be satisfied to make the output 1, that implies the **OR** logic
 - Looking at multiple rows

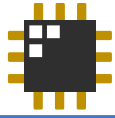
$$Q = A'B'C' + A'B'C + AB'C + ABC$$

So just do that for all the rows that has the output $Q = 1$.

Now, after getting all of those terms, we combine them by ORing the individual terms together.

So now, in English, this basically says that Q will be 1 if and only if, A is 0 and B is 0, and C is 0, OR $A = 0$ $B = 0$ $C = 1$, OR $A = 1$ $B = 0$ $C = 1$, and so on. That's exactly the English translation. And thus, this expression exactly describes this truth table.

This is already the Boolean expression that is logically equivalent to the truth table presented in this slide.



Truth tables to Boolean expression

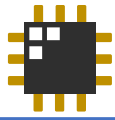
A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

- Practice: Write the equivalent Boolean expression of the truth table on the left

Here's a different truth table. Let's try to write the equivalent Boolean expression for this one.

So remember, identify all the rows that has the output equal to 1. Then, for every row, write the equivalent Boolean expression describing that row: ANDing the input conditions together as you go. Then, once you have the expressions per row, OR all of them together to create the final expression.

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$



Truth tables to Boolean expression

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

- Practice: Write the equivalent Boolean expression of the truth table on the left

$A'B'C'$

$A'B'C$

$A'BC'$

$A'BC$

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$

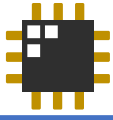
ABC'

ABC

- For the first row, A must be 0, B must be 0, and C must be 0 to make the output Q equal to 1. This is equivalent to $A'B'C'$.
- For the second row, A must be 0, B must be 0, and C must be 1 to make the output Q equal to 1. This is equivalent to $A'B'C$.
- For the third row, A must be 0, B must be 1, and C must be 0 to make the output Q equal to 1. This is equivalent to $A'BC'$.
- For the fourth row, A must be 0, B must be 1, and C must be 1 to make the output Q equal to 1. This is equivalent to $A'BC$.
- For the seventh row, A must be 1, B must be 1, and C must be 0 to make the output Q equal to 1. This is equivalent to ABC' .
- For the eighth row, A must be 1, B must be 1, and C must be 1 to make the output Q equal to 1. This is equivalent to ABC .

Combining all these terms using the OR operation, we get the Boolean expression

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$



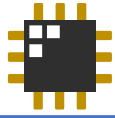
Boolean expression minimization

- Once we start writing out expressions, we start to find some 'patterns'
 - The fact that the expression resembles a math equation makes it tempting to 'group up terms' and attempt to 'simplify'...

$$Q = A'B'C' + A'B'C + AB'C + ABC$$

18

If we look closely, we can see that there are common terms that can be grouped together. It is as if we can factor out some common terms and attempt to perform some simplification. Just like in math!



Boolean expression minimization

- Once we start writing out expressions, we start to find some 'patterns'
 - The fact that the expression resembles a math equation makes it tempting to 'group up terms' and attempt to 'simplify'...

$$Q = A'B'C' + A'B'C + AB'C + ABC$$

$$Q = A'B'(C' + C) + AC(B' + B)$$

Can we do this?

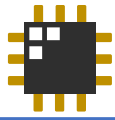
Yes, yes we can. 😊 Let's formalize some theorems first

19

For instance, in the terms $A'B'C'$ and $A'B'C$, A' and B' are common. We can factor these out, giving us $A'B'(C' + C)$.

Similarly, in the terms $AB'C$ and ABC , A and C are common. Factoring these out gives us $AC(B' + B)$.

This gives us the expression, $Q = A'B'(C' + C) + AC(B' + B)$. But, can we do this? Yes, that's the beauty of Boolean algebra. We can leverage our knowledge from math and apply that in Boolean logic as well. But before we start simplifying a lot of stuff, let's formalize some theorems first.



Boolean theorems

- For single variables: Let's say **A** is a variable

- $A(1) = A$ (Any variable ANDed to 1 is itself)
- $A + 0 = A$ (Any variable ORed to 0 is itself)
- $A(0) = 0$ (Any variable ANDed to 0 is 0)
- $A + 1 = 1$ (Any variable ORed to 1 is 1)
- $AA = A$ (Any variable ANDed to itself is itself)
- $A + A = A$ (Any variable ORed to itself is itself)
- $(A')' = A$ (Complementing twice is itself)
- $A(A') = 0$ (ANDing to its complement is 0)
- $A + A' = 1$ (ORing to its complement is 1)

AND logic

A	B	Z
0	0	0
0	1	0
1	0	0
1	1	1

OR logic

A	B	Z
0	0	0
0	1	1
1	0	1
1	1	1

20

Let's go through each of these theorems:

$A(1) = A$: This is the Identity Law for AND. It states that any variable ANDed with 1 results in the variable itself. If you look at the truth table for the AND logic, let's set B to be always 1, so now, as long as the other operand is fixed to 1, the output just copies whatever A is. Thus, it is true that $A(1) = A$.

$A + 0 = A$: This is the Identity Law for OR. It states that any variable ORed with 0 results in the variable itself. If you look at the truth table for the OR logic, let's set B to be always 0, so now, as long as the other operand is fixed to 0, the output just copies whatever A is. Thus, it is true that $A + 0 = A$.

$A(0) = 0$: This is the Null Law for AND. It states that any variable ANDed with 0 results in 0. If you look at the truth table for the AND logic, let's set B to be always 0, so now, as long as the other operand is fixed to 0, the output will always be 0 regardless of what A is. Thus, it is true that $A(0) = 0$.

$A + 1 = 1$: This is the Null Law for OR. It states that any variable ORed with 1 results in 1. If you look at the truth table for the OR logic, let's set B to be always 1, so now, as long as the other operand is fixed to 1, the output will always be 1 regardless of what A is. Thus, it is true that $A + 1 = 1$.

$AA = A$: This is the Idempotent Law for AND. It states that any variable ANDed with

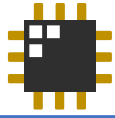
itself results in the variable itself. If you look at the truth table for the AND logic, let's set B to be always equal to A, so now, as long as the other operand is the same as A, the output just copies whatever A is. Thus, it is true that $AA = A$.

$A + A = A$: This is the Idempotent Law for OR. It states that any variable ORed with itself results in the variable itself. If you look at the truth table for the OR logic, let's set B to be always equal to A, so now, as long as the other operand is the same as A, the output just copies whatever A is. Thus, it is true that $A + A = A$.

$(A')' = A$: This is the Law of Double Negation. It states that the complement of the complement of a variable is the variable itself. If you look at the truth table for the NOT logic, let's apply the NOT operation twice on A, so now, the output just copies whatever A is. Thus, it is true that $(A')' = A$.

$A(A') = 0$: This is the Complement Law for AND. It states that a variable ANDed with its complement results in 0. If you look at the truth table for the AND logic, let's set B to be always the complement of A, so now, as long as the other operand is the complement of A, the output will always be 0. Thus, it is true that $A(A') = 0$.

$A + A' = 1$: This is the Complement Law for OR. It states that a variable ORed with its complement results in 1. If you look at the truth table for the OR logic, let's set B to be always the complement of A, so now, as long as the other operand is the complement of A, the output will always be 1. Thus, it is true that $A + A' = 1$.



Boolean theorems

- For multiple variables: Let's say **A**, **B**, and **C** are variables

- $AB = BA, A+B = B+A$ (Commutativity)

- $(AB)C = A(BC), (A+B)+C = A+(B+C)$ (Associativity)

- $A(B+C) = AB + AC$ (Distributivity)

Feels like math so far...

- $A+(BC) = (A+B)(A+C)$ (Distributivity, but weirder!)

We can also distribute the complement! (DeMorgan's theorem)

- $(AB)' = A' + B'$; another version is $(A+B)' = A'B'$

(Take note of the change in operator!)

21

These are some of the fundamental theorems of Boolean algebra that are indeed similar to the laws in arithmetic. Let's go through them:

1. $AB = BA, A+B = B+A$: These are the Commutative Laws. They state that the order of the variables does not matter in an AND or OR operation, just like addition or multiplication in arithmetic.

2. $(AB)C = A(BC), (A+B)+C = A+(B+C)$: These are the Associative Laws. They state that the way variables are grouped in an AND or OR operation does not matter, similar to addition or multiplication in arithmetic.

3. $A(B+C) = AB + AC$: This is the Distributive Law. It states that AND distributes over OR, similar to how multiplication distributes over addition in arithmetic.

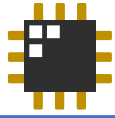
4. $A+(BC) = (A+B)(A+C)$: This is another form of the Distributive Law. It's a bit less intuitive than the previous one since it doesn't work like this in math.

What's really happening is that the OR operator is being distributed over the AND operator. This is legit though, and we can prove that this is indeed true.

$$(A+B)(A+C) = (AA + AC + BA + BC) = (A + AC + AB + BC) = A(1+C+B) + BC = A(1) + BC = A+BC.$$

5. $(AB)' = A' + B'$, $(A+B)' = A'B'$: These are De Morgan's Laws. They state that the complement of an AND operation is the same as the OR of the

complements, and vice versa. The AND operation changes to OR (and vice versa) when taking the complement. This is a unique property of Boolean algebra and doesn't have a direct equivalent in arithmetic.



Boolean minimization

- Practice for a bit, write a simplified expression for these terms

- $A(A+B) = ?$

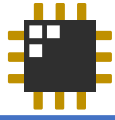
- $AB + AB' = ?$

22

Let's simplify these expressions using the Boolean algebra theorems:

1. $A(A+B) = ?$ We can use the Distributive Law here, which states that $A(B+C) = AB + AC$. Applying this law to the expression $A(A+B)$, we get $AA + AB$. But we know that $(AA = A)$, the term AA can be simplified to A . So, we have $A + AB$. Continuing further, we can factor out A , we have: $A(1+B)$. But we know that $1+B = 1$. Therefore, we end up with $A(1) = A$. The simplified expression is just A .

2. $AB + AB' = ?$ We can factor out the common term A , so we end up with $A(B+B')$. But we know that $B+B' = 1$. Therefore, we have $A(1) = A$. The simplified expression is just A .

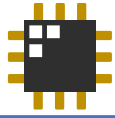


Boolean minimization

- Practice for a bit, write a simplified expression for these terms
 - $A(A+B) = ?$
 - $AA + AB$
 - $A + AB$
 - $A(1+B)$
 - A
 - $AB + AB' = ?$

23

For this first one, we can perform distribution, putting the variable A inside the parenthesis. We will end up with $AA + AB$. But we know that $(AA = A)$, therefore the term AA can be simplified to A . So, we have $A + AB$. Continuing further, we can factor out A, giving us $A(1+B)$. But we know that anything ORed to 1 is just 1 from the previous theorems before, therefore $1+B = 1$. We end up with $A(1) = A$. This is the simplified expression.

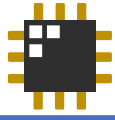


Boolean minimization

- Practice for a bit, write a simplified expression for these terms
 - $A(A+B) = ?$
 - $AA + AB$
 - $A + AB$
 - $A(1+B)$
 - A
 - $AB + AB' = ?$
 - $A(B+B')$
 - $A(1)$
 - A

24

For the second expression, we can simply factor out the common term A , so we end up with $A(B+B')$. But we know that $B+B' = 1$ from the previous theorems before. Therefore, we have $A(1) = A$. The simplified expression is just A .



Boolean expression minimization (Part 1)

- Let's go back to this!

$$Q = A'B'C' + A'B'C + AB'C + ABC$$

$$Q = A'B'(C' + C) + AC(B' + B)$$

Can we do this?

Yes, yes we can. 😊

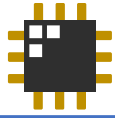
25

So now, let's go back to this expression.

We identified earlier that $A'B'$ are common for the 1st and 2nd terms and we factored that out.

We also identified that AC are common for the 3rd and 4th terms and we factored that out.

So yes, this is indeed possible given the theorems we have presented before.



Boolean expression minimization (Part 1)

$$Q = A'B'C' + A'B'C + AB'C + ABC$$

$$Q = A'B'(C' + C) + AC(B' + B)$$

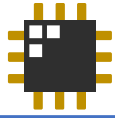
$$Q = A'B'(1) + AC(1)$$

$$Q = A'B' + AC$$

Final answer

26

Let's continue further. After factoring out $A'B'(C' + C) + AC(B' + B)$. But we know that $C' + C = 1$. Also, $B' + B = 1$. Therefore, we end up with $A'B' + AC$.



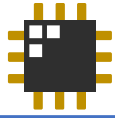
Boolean expression minimization (Part 2)

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$

27

Let's have another practice. This is a different Boolean expression from before.

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$



Boolean expression minimization (Part 2)

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$

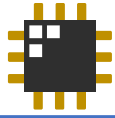
$$Q = A'B'(C' + C) + A'B(C' + C) + AB(C' + C)$$

$$Q = A'B' + A'B + AB$$

28

$Q = A'B'(C' + C) + A'B(C' + C) + AB(C' + C)$ Here, we're using the Distributive Law in reverse to factor out common terms. In each group of terms, $A'B'$, $A'B$, and AB are common, so we factor them out.

$Q = A'B' + A'B + AB$ Here, we're using the Identity Law for OR, which states that any variable ORed with its complement results in 1. So, $(C' + C)$ simplifies to 1, and any variable ANDed with 1 is itself, so $A'B'(1)$ simplifies to $A'B'$, and so on.



Boolean expression minimization (Part 2)

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$

$$Q = A'B'(C' + C) + A'B(C' + C) + AB(C' + C)$$

$$Q = A'B' + A'B + AB$$

$$Q = A'B' + A'B + A'B + AB$$

$$Q = A'B' + A'B + A'B + AB$$

$$Q = A'(B' + B) + B(A' + A)$$

$$Q = A' + B \quad \text{Final answer}$$

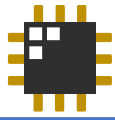
29

$Q = A'B' + A'B + A'B + AB$ Here, we replicate $A'B$. This is fine because we know that any expression ORed to itself is itself. Replicating $A'B$ is also helpful because we can perform further factoring out, given the common terms A' and B on the first and last terms.

$Q = A'(B' + B) + B(A' + A)$ Here, we're using the Distributive Law in reverse again to factor out common terms. In the first three terms, A' is common, so we factor it out. In the last two terms, B is common, so we factor it out.

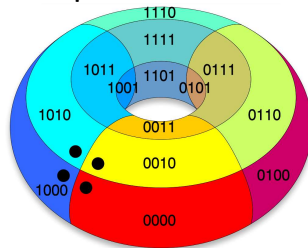
$Q = A' + B$ Finally, we're using the Identity Law for OR again to simplify $(B' + B)$ and $(A' + A)$ to 1. So, $A'(1)$ simplifies to A' , and $B(1)$ simplifies to B .

So, the original Boolean expression $Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$ simplifies to $Q = A' + B$. This is the power of Boolean algebra in simplifying complex logical expressions!



Boolean minimization

- How do we know if it's already in the most simplified expression?
 - There are techniques taught in EEE 143 that will allow you to confirm whether it cannot be simplified further... We won't cover it here.



0000	0100	1100	1000
0001	0101	1101	1001
0011	0111	1111	1011
0010	0110	1110	1010

It looks like this...

Though probably not drawn as a doughnut ☹

- When we give minimization problems like this, we'll make sure the 'patterns' are easy to find. ☺
- We'll also try not to give a very long expression to start with.

30

Image credit: "Karnaugh map" wikipedia.org

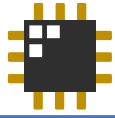
A Boolean expression is considered to be in its simplest form when it cannot be reduced any further, meaning no more identities, properties, or theorems of Boolean algebra can be applied to simplify the expression.

Here are some indicators that a Boolean expression is in its simplest form:

1.No Redundant Terms: The expression does not contain any redundant terms. Redundant terms are those that do not change the output of the expression whether they are included or not.

2.No Common Factors: The expression does not have any common factors that can be factored out using the distributive property.

3.No Complement Pairs: The expression does not contain any complement pairs. A complement pair is a variable and its complement. For example, in the term $(A + A')$, A' is the complement of A . According to the Complement Law, $A + A'$ simplifies to 1.



Logical equivalence

- Are we sure that the simplified expression is equivalent to the original expression?
 - As long as you followed the theorems correctly, it should 😊



- You can write the truth table again using the simplified expression. The resulting truth table should be the same!

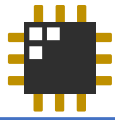
GIF from tenor.com

31

Of course, the next question is, after doing all the simplification that we did, are we still sure that the simplified expression is equivalent to the original expression?

Well, as long as you followed the theorems correctly, and you are not careless when applying them, then yes, they should be equivalent.

One way to verify the equivalence is to write the truth tables for both original and simplified expressions. The resulting truth table should be the same!



Logical equivalence (Part 1)

$$Q = A'B'C' + A'B'C + AB'C + ABC$$

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

$$Q = A'B' + AC$$

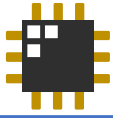
A	B	C	Q
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

32

So the table on the left is the original truth table. If you can remember, we actually started from the truth table then derived the expression from it. So now, let's try to write the truth table for the simplified expression.

$A'B'$ = as long as A is 0 and B is 0 (doesn't matter what C is), the output becomes 1

AC = as long as A is 1 and C is 1 (doesn't matter what B is), the output becomes 1



Logical equivalence (Part 1)

$$Q = A'B'C' + A'B'C + AB'C + ABC$$

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

*Tables are the same
Logically equivalent!*

$$Q = A'B' + AC$$

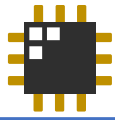
A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

33

$A'B'$ = as long as A is 0 and B is 0 (doesn't matter what C is), the output becomes 1 (orange highlight)

AC = as long as A is 1 and C is 1 (doesn't matter what B is), the output becomes 1 (blue highlight)

As you can see, the truth table is exactly the same. Therefore, both expressions are logically equivalent!



Logical equivalence (Part 2)

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

$$Q = A' + B$$

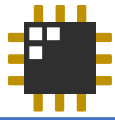
A	B	C	Q
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

34

Here's the other expression that we derived and the corresponding truth table. Let's verify that the simplified expression is indeed the same as the original expression.

A' = as long as A is 0, doesn't matter what B and C are, the output is 1.

B = as long as B is 1, doesn't matter what A and C are, the output is 1.



Logical equivalence (Part 2)

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

*Tables are the same
Logically equivalent!*

$$Q = A' + B$$

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

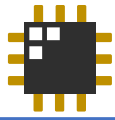
35

A' = as long as A is 0, doesn't matter what B and C are, the output is 1 (orange highlight)

B = as long as B is 1, doesn't matter what A and C are, the output is 1. (blue highlight)

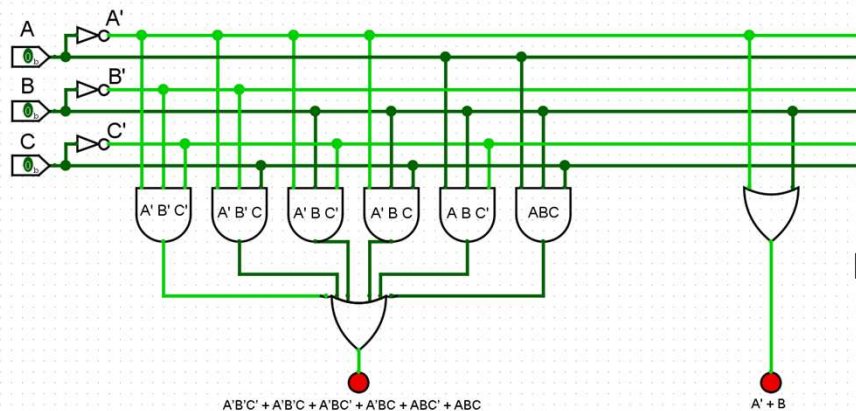
We also note that there's an intersection between the orange and blue highlights. Meaning the two terms we are considering has two rows in the truth table that they both correspond to. This is fine, since they both map to an output of 1 anyway.

As we can see, the resulting truth table for the simplified expression is indeed the same as the truth table of the original expression, thus proving their equivalence!



Logical equivalence

- If the truth tables are the same, they are functionally equivalent!
- Simulation demo:



**Fewer logic gates.
More efficient!**

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$

$$Q = A' + B$$

36

As a final note, we can also perform some simulations to really verify their equivalence. I have a simulator here where you can see the expressions being wired up. You can see the AND gates, OR gates, and NOT gates in there that exactly corresponds to how the expressions describe it. Try observing the connections on your own to verify!

On the left is the original expression and its corresponding logic diagram. On the right is the simplified expression and its corresponding diagram. (Simulator: note that when we are changing the inputs to any possible combination, the outputs shown in red behave similarly. This clearly indicates that they behave exactly the same. This further proves that they are logically equivalent.)

Moreover, as you can see, the simplified expression has lower number of logic gates. This actually alludes to that being more efficient in terms of hardware, since you can implement the same logic functionality while having fewer logic gates to do it. That is the ultimate benefit of Boolean logic simplification.